Escucha social con IA, sin licencia

El paso a paso completo para clasificar 20.000 menciones de una marca por unos US$ 6: captura con APIs, embudo, set dorado, Gemini Flash, imágenes, medición y un informe que se verifica solo. Con el código.

Jonathan Rodríguez
Construir herramientas con IA, sin ser programador
linkedin.com/in/jonathanrod1986/

Antes de empezar

Clasificar 20.000 menciones de una marca con un modelo de IA barato cuesta hoy unos 6 dólares. Una licencia de escucha social cuesta miles al año. Y en los dos casos casi nadie mide lo único que de verdad importa: **si el sentimiento que le reportan a la marca es correcto**.

Esta guía es el flujo que corro, completo, para que lo puedas replicar. Son ocho pasos, cada uno deja un archivo, y en cada uno está el código o la plantilla que necesitas. Lo aprendí en estudios reales, varias veces a las malas, y aquí van también los errores.

Para quién es

- Para quien lidera marketing o marca y quiere saber qué pedirle a su equipo (o a su agencia) cuando le reportan «el sentimiento».
- Para el estratega o analista que lo va a correr. Esta guía tiene más código que las anteriores: son programas cortos en Python, y cada bloque va explicado. Si nunca has corrido un script, pídele a tu asistente de IA que te acompañe con el primero; después es copiar y adaptar.

Qué te llevas

- El flujo de ocho pasos con el archivo que sale de cada uno.
- Código para capturar comentarios de YouTube con la API oficial, armar el embudo, clasificar con Gemini con tope de gasto, leer el tono con modelos abiertos, revisar imágenes y medir contra el set dorado.
- El prompt completo, la taxonomía y las reglas para que el tono no te engañe.
- La cuenta de cuánto cuesta, con números.
- Una lista de verificación y la plantilla de la hoja de vida del estudio.

El ejemplo que vamos a usar

Una tienda en línea de café de Bogotá (ficticia, la misma de la guía 01) quiere saber **de qué habla la gente que compra café en línea**: qué le gusta, de qué se queja y qué tanto pesa cada cosa. Junta tres meses de conversación: comentarios de videos de reseñas de café en YouTube, posts de X y las reseñas de su propia tienda. Al final del embudo le quedan unas **20.000 menciones** para clasificar y unas **2.000 imágenes**.

Cambia «café» por tu categoría y el método sirve igual.

Lo que necesitas

Qué	Para qué	Costo
Un proyecto en Google Cloud con la YouTube Data API v3 activada y una clave	Capturar comentarios de YouTube	Gratis dentro de la cuota diaria
Una clave de la API de Gemini	Clasificar texto e imágenes	Pago por uso
Python 3 con `pandas` y `requests`	Correr los pasos	Gratis
Opcional: la API de X	Posts de X	US$ 0,005 por post leído
Opcional: `pysentimiento` y `transformers`	Lectores de tono abiertos, en tu computador	Gratis

> **Ojo con las claves:** guárdalas en un archivo `.env` fuera de cualquier carpeta que compartas y léelas desde el programa. Nunca las pegues dentro del código ni en un documento.

Lo que esto no te da (y una licencia sí)

Históricos de años, cobertura completa de redes y medios en tiempo real, y un tablero listo para usar con soporte detrás. Si tu pregunta necesita eso, paga la licencia. Esta guía sirve para la mayoría de preguntas de marca que se responden con unos meses de conversación y un buen método. Y lo que aprendes aquí (el set dorado, sobre todo) te sirve también para auditar lo que te entrega la licencia.

PASO 1

Encuadre: la pregunta antes que los datos

El error más caro de una escucha no está en la IA. Está en la pregunta. Un estudio que arranca con «¿hablan mal de nosotros?» termina contando quejas, porque eso fue lo que se le pidió.

Antes de abrir cualquier dato escribo tres cosas:

- **La pregunta real y su fase.** Muchos encargos tienen dos tiempos: primero se describe de qué habla la gente y después se decide algo (qué comunicar, qué arreglar, dónde invertir). Si se mezclan, el estudio juzga antes de entender.
- **Las hipótesis del cliente, anotadas como supuestos.** Se contrastan al final; no encuadran el estudio.
- **Lo que la escucha no puede decir.** Recordación, ventas, cuánta gente piensa así. La conversación en redes no es una muestra de la población, y conviene decirlo el primer día.

PLANTILLA · ENCUADRE.MD

pregunta: ¿De qué hablan quienes compran café en línea en Bogotá? fase: describir (decidir qué comunicar viene en la fase 2) supuesto: «se quejan del precio» (se contrasta al final) ventana: 1 de julio a 30 de septiembre no dice: recordación, ventas ni cuánta gente piensa así

PASO 2

Captura: cada fuente hace un trabajo distinto

No todas las fuentes sirven para lo mismo. Las separo por el trabajo que hacen:

Trabajo	Qué responde	Con qué
Escucha de la categoría	De qué se habla, en general	YouTube (API oficial), X (API oficial), prensa
Censo de una campaña	Todos los comentarios de una pieza o un lanzamiento	La API de la red, o Apify con autorización
Cuentas propias	Lo que le escriben a la marca	Las herramientas de la propia red o del e-commerce

YouTube con la API oficial

La YouTube Data API v3 tiene una cuota diaria gratuita. Lo que importa saber: **buscar videos** (`search.list`) cuesta 100 unidades por llamada, y **traer comentarios** (`commentThreads.list`) cuesta 1 unidad y trae hasta 100 comentarios por llamada. Así que conviene buscar poco y traer mucho: haz una lista corta de videos relevantes (reseñas de café, comparaciones de marcas, tutoriales de preparación) y trae todos sus comentarios.

```
import hashlib, json, os, re, requests

def leer_clave(nombre, ruta=".env"):
    # Lee una clave del .env sin ejecutarlo
    for linea in open(ruta, encoding="utf-8"):
        m = re.match(rf"^{nombre}=(.*)$", linea.strip())
        if m:
            return m.group(1).strip().strip('"')
    raise SystemExit(f"Falta {nombre} en {ruta}")

CLAVE = leer_clave("YOUTUBE_API_KEY")
URL = "https://www.googleapis.com/youtube/v3/commentThreads"

def anonimo(autor):
    # El autor nunca viaja en claro: un código corto e irreversible
    return hashlib.sha256(autor.encode()).hexdigest()[:10]

def comentarios(video_id, salida):
    pagina = None
    while True:
        params = {"part": "snippet", "videoId": video_id, "maxResults": 100,
                  "textFormat": "plainText", "key": CLAVE}
        if pagina:
            params["pageToken"] = pagina
        r = requests.get(URL, params=params, timeout=30)
        r.raise_for_status()
        datos = r.json()
        for item in datos.get("items", []):
            s = item["snippet"]["topLevelComment"]["snippet"]
            fila = {"k": "yt-" + item["id"], "red": "youtube",
                    "fecha": s["publishedAt"][:10], "texto": s["textOriginal"],
                    "url": f"https://www.youtube.com/watch?v={video_id}",
                    "autor": anonimo(s.get("authorDisplayName", ""))}
            salida.write(json.dumps(fila, ensure_ascii=False) + "\n")
        pagina = datos.get("nextPageToken")
        if not pagina:
            break

with open("menciones.jsonl", "a", encoding="utf-8") as f:
    for v in ["ID_DEL_VIDEO_1", "ID_DEL_VIDEO_2"]:
        comentarios(v, f)
```

Qué hace: pide los comentarios de cada video de a 100, sigue la página siguiente hasta que no haya más y escribe una línea por comentario en `menciones.jsonl`. El nombre del autor se cambia por un código: para analizar conversación no necesitas saber quién es cada persona, y así no cargas datos personales.

X y las reseñas propias

- **X** tiene API oficial de pago por uso: US$ 0,005 por post leído. 50.000 posts son unos US$ 250, así que aquí sí conviene afinar bien la búsqueda antes de traer nada.
- **Las reseñas de tu propia tienda** se exportan desde el panel de tu e-commerce o de tu perfil de negocio. Son tuyas, traen la calificación y tienen un sesgo conocido: escriben más los muy felices y los muy bravos.

Apify y las redes que no lo permiten

Apify tiene un «actor» (un programa de extracción) para casi cada red, y cobra por resultado. Úsalo donde los términos de la red lo permiten.

> **Ojo:** extraer Instagram, Facebook o TikTok con Apify va contra los términos de esas redes **aunque el contenido sea público**. Solo se hace con autorización del dueño de la cuenta, sin iniciar sesión, sin datos personales en el entregable y dejándolo escrito en la hoja de vida del estudio. En Colombia, el tratamiento de datos personales se rige por la Ley 1581 de 2012. Las herramientas académicas de esas redes no sirven para trabajo comercial.

Al final de la captura tienes un solo archivo, `menciones.jsonl`, con una línea por mención:

```
{"k": "yt-UgzX81", "red": "youtube", "fecha": "2026-08-14",
 "texto": "El café llegó frío y tres días tarde...",
 "url": "https://www.youtube.com/watch?v=...", "autor": "a7f3c91b02"}
```

PASO 3

Embudo: cada filtro con su conteo

Antes de contar nada hay que saber qué tipo de cosa es cada fila. No todo lo que capturas es conversación de personas, y mezclarlo distorsiona el tono: el comercio electrónico es positivo por defecto («¡envío gratis!») y los medios son neutrales. En un estudio que corrí, sin esta separación, el ranking de sentimiento entre países salió al revés.

Nivel	Criterio	Se usa para
descartar	Vacío, duplicado exacto, ruido (series, videojuegos, sentido figurado)	Nada
comercio electrónico	Tiendas y marketplaces	Precio y surtido; nunca tono
medios	Notas de prensa y portales	Agenda; no es voz de personas
débil	Menos de 25 caracteres o sin palabras de la categoría	Respaldo; no se cita
conversación	Todo lo demás	El análisis

Dos señales que ayudan en la misma pasada: un autor con más del 0,5 % de las filas casi siempre es un medio, una tienda o un bot; y lo que sale del corte temático **es una capa, no basura**. En otro estudio, justo la capa que el corte había dejado por fuera era donde vivía la conversación emocional, el hallazgo que el cliente terminó valorando más. Se guarda con nombre.

El embudo en Python

```
import json, re
import pandas as pd

df = pd.read_json("menciones.jsonl", lines=True)
pasos = [("0 captura", len(df))]

def limpio(t):
    t = re.sub(r"https?://\S+|@\w+", " ", str(t))
    return re.sub(r"\s+", " ", t).strip()

df["limpio"] = df["texto"].map(limpio)
TIENDAS = re.compile(r"env[ií]o gratis|compra ya|descuento|oferta|tienda oficial", re.I)
CATEGORIA = re.compile(r"caf[eé]|tueste|grano|molido|espresso|cafetera|taza", re.I)

def nivel(t):
    if len(t) == 0:
        return "descartar"
    if TIENDAS.search(t):
        return "comercio"
    if len(t) < 25 or not CATEGORIA.search(t):
        return "debil"
    return "conversacion"

df["nivel"] = df["limpio"].map(nivel)
df = df.drop_duplicates(subset=["red", "limpio"])
pasos.append(("1 sin duplicados", len(df)))
conv = df[df["nivel"] == "conversacion"]
pasos.append(("2 conversación", len(conv)))

autores = conv["autor"].value_counts(normalize=True)
dominantes = set(autores[autores > 0.005].index)
conv = conv[~conv["autor"].isin(dominantes)]
pasos.append(("3 sin autores dominantes", len(conv)))

conv.to_json("conversacion.jsonl", orient="records", lines=True, force_ascii=False)
json.dump({"pasos": pasos, "por_nivel": df["nivel"].value_counts().to_dict()},
          open("embudo.json", "w"), ensure_ascii=False, indent=1)
for nombre, n in pasos:
    print(f"{nombre:28} {n:>8,}".replace(",", "."))
```

Qué hace: quita enlaces y menciones, marca el nivel de cada fila con reglas simples, saca duplicados, se queda con la conversación, aparta a los autores dominantes y deja escrito cuántas filas sobrevivieron a cada paso. Las palabras de `TIENDAS` y `CATEGORIA` son del ejemplo: ajústalas a tu categoría leyendo una muestra.

Así se ve la salida (cifras de ejemplo):

```
0 captura                    31.200
1 sin duplicados             28.940
2 conversación               21.310
3 sin autores dominantes     20.150
```

> **Por qué importa el conteo:** en cuanto alguien ve el primer tablero pregunta de dónde salen las menciones. Si cada paso tiene criterio, código y conteo, cualquier cifra se puede seguir hasta la mención cruda. Un paso hecho a mano y sin registro vuelve discutible todo lo que viene después.

PASO 4

Taxonomía cerrada y set dorado

La taxonomía, en capas

Una lista cerrada de temas es lo que hace que la clasificación se pueda comparar y validar. La armo en capas: **de 3 a 5 pilares** (las lecturas grandes con las que decide la marca) y **de 12 a 20 etiquetas** debajo, varias por mención. Cada etiqueta lleva su definición **y lo que NO es**. El «no» es lo que más sube la precisión: le dice al modelo dónde está la frontera.

Además de los temas, cada mención lleva cuatro campos:

- `rel`: si de verdad habla del tema o solo comparte una palabra.
- `tono`: positivo, negativo o neutral.
- `objeto`: **sobre qué recae el tono** (el producto, la entrega, el precio, la atención, la marca, ninguno). «Me encanta el café, pero la entrega fue un desastre» es negativa sobre la entrega, no sobre el café.
- `concreto`: si alude sin ambigüedad a un pedido, una campaña o un producto concreto.

PLANTILLA · TAXONOMIA.JSON (EXTRACTO DEL EJEMPLO)

{"version": "1.0 · 2026-10-01", "regla": "Se etiqueta lo que la mención dice, no lo que insinúa. El tono se juzga sobre el objeto, no sobre el ánimo general. La ironía se lee por lo que quiere decir.", "pilares": [{"id": "A", "nombre": "El café", "etiquetas": [{"id": "A1", "nombre": "Sabor y frescura", "def": "Cómo sabe, si llegó fresco, notas, tueste", "no": "El empaque que mantiene la frescura va en B2"}, {"id": "A2", "nombre": "Variedad", "def": "Orígenes, tipos de grano, molido", "no": "El precio de una variedad va en C1"}]}, {"id": "B", "nombre": "La entrega", "etiquetas": [{"id": "B1", "nombre": "Demora", "def": "Llegó tarde o no llegó: «tres días», «nunca llegó»", "no": "El costo del envío no es demora: eso es C2"}, {"id": "B2", "nombre": "Empaque", "def": "Cómo llegó la bolsa o la caja", "no": "El diseño de la marca va en D1"}]}, {"id": "C", "nombre": "Precio y pago", "etiquetas": [{"id": "C1", "nombre": "Precio", "def": "Caro, barato, vale lo que cuesta", "no": "Descuentos de tienda: se descartan en el embudo"}, {"id": "C2", "nombre": "Costo de envío", "def": "Cuánto cobran por enviar", "no": "La demora va en B1"}]}]}

Guarda versiones: fecha, número y qué cambió. Cuando una etiqueta se redefine, lo etiquetado con la definición vieja deja de valer para ella.

El set dorado

Es la lectura humana contra la que se mide todo. Sin él, ninguna cifra de sentimiento tiene respaldo.

- **Tamaño:** entre 300 y 500 menciones. En el ejemplo, 400.
- **Estratificado:** cuota por red (no proporcional al volumen), y **más casos dudosos** de los que habría al azar: por ejemplo 30 % que parecen negativos, 25 % ambiguos, 25 % positivos y 20 % neutrales.
- **Barajado** y leído, en lo posible, **sin ver la etiqueta del modelo**: verla ancla la lectura.
- **Dos lectores** sobre 50 a 100 menciones, y se mide su acuerdo. Si dos personas no coinciden, el modelo tampoco va a poder.
- **Partido en dos mitades:** con una ajustas el prompt y con la otra reportas. Si ajustas y reportas con las mismas 400, las métricas salen infladas.

PLANTILLA · DORADO.CSV

k, red, texto, rel_humano, temas_humano, tono_humano, objeto_humano, lector, mitad yt-UgzX81, youtube, "El café llegó frío y tres días tarde", si, B1, negativo, la entrega, JR, reporte

PASO 5

Texto: el modelo etiqueta, la lista manda

El prompt

El prompt de sistema se arma **desde el JSON de la taxonomía**, con código, nunca a mano. Así, cuando cambias una definición, cambia en un solo lugar.

```
Eres analista de escucha social especializado en café en Colombia.
Vas a etiquetar menciones en español con esta taxonomía. Responde SOLO con JSON.
REGLA: se etiqueta lo que la mención dice, no lo que insinúa; el tono se juzga
sobre el objeto del juicio; la ironía se lee por lo que quiere decir.
ETIQUETAS DE TEMA (varias por mención; lista vacía si no aplica ninguna):
Pilar A · El café
  A1 Sabor y frescura: cómo sabe, si llegó fresco. NO: el empaque va en B2
  A2 Variedad: orígenes, tipos de grano. NO: el precio va en C1
Pilar B · La entrega
  B1 Demora: llegó tarde o no llegó. NO: el costo del envío es C2
  ...
TONO (Positivo/Negativo/Neutral): valoración explícita o claramente implícita.
OBJETO (el café, la entrega, el precio, la atención, la marca, ninguno).
REL (true/false): habla de café comprado o tomado, no solo de la palabra.
CONCRETO (true/false): alude a un pedido, producto o campaña concretos.
Si rel es false: temas [], tono Neutral, objeto ninguno, concreto false.
FORMATO: un array JSON, un objeto por mención, en el mismo orden:
{"k": <id>, "rel": bool, "temas": [...], "tono": "...", "objeto": "...", "concreto": bool}
```

El mensaje de usuario es «Etiqueta estas 25 menciones:» y una línea por mención, `[k] texto`, con el texto recortado a unos 420 caracteres, sin enlaces ni @menciones.

Los parámetros que funcionaron

Parámetro	Valor	Por qué
Modelo	Un modelo «flash» del proveedor	Calidad suficiente a una fracción del costo
Temperatura	0	Que dé lo mismo cada vez
Salida	JSON estricto	Se lee sin adivinar
Razonamiento	Apagado	En mi validación costó la cuarta parte y no perdió calidad. Compruébalo contra tu dorado
Lote	25 menciones por llamada	Equilibrio entre repetir el prompt y respuestas ilegibles
Hilos	4	Sin chocar con los límites de la API

La llamada, con tope de gasto

```
import json, time, requests

CLAVE = leer_clave("GEMINI_API_KEY")          # la función del paso 2
MODELO = "gemini-flash-latest"                # el modelo flash vigente
URL = f"https://generativelanguage.googleapis.com/v1beta/models/{MODELO}:generateContent"
# Precios en US$ por millón de tokens: pon los del día.
# Si los dejas en 0, el tope no protege nada.
PRECIO_ENTRADA, PRECIO_SALIDA = 0.0, 0.0
TOPE_US = 10.0
gastado = 0.0

def etiquetar(lote, sistema):
    global gastado
    texto = "Etiqueta estas %d menciones:\n" % len(lote) + "\n".join(
        f"[{m['k']}] {m['limpio'][:420]}" for m in lote)
    cuerpo = {"systemInstruction": {"parts": [{"text": sistema}]},
              "contents": [{"role": "user", "parts": [{"text": texto}]}],
              "generationConfig": {"temperature": 0,
                                   "responseMimeType": "application/json",
                                   "maxOutputTokens": 8192,
                                   "thinkingConfig": {"thinkingBudget": 0}}}
    for intento in range(4):
        r = requests.post(URL, params={"key": CLAVE}, json=cuerpo, timeout=120)
        if r.status_code == 402:
            raise SystemExit("Sin crédito: se corta aquí, sin perder lo hecho.")
        if r.status_code in (429, 500, 503):
            time.sleep(2 ** intento * 5)
            continue
        r.raise_for_status()
        datos = r.json()
        uso = datos.get("usageMetadata", {})
        gastado += (uso.get("promptTokenCount", 0) * PRECIO_ENTRADA
                    + uso.get("candidatesTokenCount", 0) * PRECIO_SALIDA) / 1e6
        try:
            return json.loads(datos["candidates"][0]["content"]["parts"][0]["text"])
        except (KeyError, json.JSONDecodeError):
            continue                                # un reintento si llega roto
    return None                                     # el lote queda sin etiquetar y se cuenta
```

Y el ciclo que lo usa, reanudable:

```
hechas = set()
try:
    hechas = {json.loads(l)["k"] for l in open("etiquetas.jsonl", encoding="utf-8")}
except FileNotFoundError:
    pass
pendientes = [m for m in conversacion if m["k"] not in hechas]
with open("etiquetas.jsonl", "a", encoding="utf-8") as salida:
    for i in range(0, len(pendientes), 25):
        if gastado >= TOPE_US:
            print("Tope alcanzado: no sale ninguna llamada más."); break
        res = etiquetar(pendientes[i:i + 25], sistema)
        for fila in res or []:
            salida.write(json.dumps(fila, ensure_ascii=False) + "\n")
```

Tres detalles que valen plata: cada mención se escribe **en cuanto llega**, así que si algo se cae, al volver a correr se salta lo hecho y no pagas dos veces. El gasto se calcula con los tokens **que factura la API**, no con una estimación. Y al llegar al tope no sale ninguna llamada más.

> **Ojo con el razonamiento:** según la versión del modelo, el parámetro para apagarlo cambia de nombre (en unos es `thinkingBudget: 0`, en otros un nivel bajo de razonamiento). Revisa la documentación del modelo que uses; en los que facturan el razonamiento como salida, es el grueso del costo.

La sonda: el costo antes de gastar

Antes de correr las 20.000, corro **de 4 a 6 lotes reales**, sumo los tokens que facturó la API y proyecto al total. Con eso escribo cuatro números: universo, costo proyectado, tiempo estimado y tope. Quien paga lo aprueba antes, no después. Lo etiquetado en la sonda no se tira: ya está en `etiquetas.jsonl`.

Concepto	Por mención	20.000 menciones
Tokens de entrada (con el prompt repartido)	~145	~2,9 millones
Tokens de salida	~50	~1,0 millón
Costo observado en oct-2026		**~US$ 6**

Ese costo es con la tarifa promocional de Gemini Flash vigente hasta el 31-dic-2026; desde 2027 será cerca del doble, o lo mismo usando el modo por lotes (Batch). Haz siempre tu cuenta con los precios del día.

Dos pasadas: la relevancia se decide en los dos sentidos

Si antes del modelo usas un filtro barato (reglas, como las del paso 3), ese filtro tira menciones buenas antes de que el modelo las vea. En un estudio real perdía una parte nada despreciable de las relevantes del dorado, justo algunas de las citas más valiosas. Por eso:

1. **Primera pasada** sobre lo que el filtro dio por relevante: el modelo puede sacar falsos positivos.
2. **Segunda pasada** sobre lo que el filtro descartó, con su propio costo aprobado: el modelo puede rescatar falsos negativos.
3. Mide en el dorado cuántas recupera la segunda pasada. Es el argumento para pagarla.

Que el tono no te engañe

El sentimiento que traen las plataformas marca casi todo como neutral. Reclasificado, el negativo puede multiplicarse varias veces. Por eso el tono lo deciden **varios lectores**, y dos de ellos corren gratis en tu computador:

```
from pysentimiento import create_analyzer
from transformers import pipeline

tono_es = create_analyzer(task="sentiment", lang="es")        # entrenado en redes en
español
ironia = create_analyzer(task="irony", lang="es")
tono_multi = pipeline("sentiment-analysis",
                      model="cardiffnlp/twitter-xlm-roberta-base-sentiment")

texto = "Qué maravilla, otra vez el pedido sin llegar"
print(tono_es.predict(texto).output, ironia.predict(texto).output,
      tono_multi(texto)[0]["label"])
```

Las reglas para decidir, en este orden:

1. Si los lectores coinciden, ese es el tono.
2. Si discrepan, **manda el del idioma** (en español, el de `pysentimiento`). La regla «si discrepan, neutral» la probé: corregía tantos errores como aciertos rompía.
3. Donde leyó el modelo grande del paso 5, **manda él**: contra el dorado acertaba más que las reglas.
4. Lo informativo sin juicio (una noticia, una promoción) es neutral.
5. **Lo negativo necesita objeto.** «Me duele todo» no es una queja contra nadie.

Y la **regla del humor**, que se aplica al final: una mención marcada negativa, con humor, sin queja concreta y sin evento, pasa a neutral. Los modelos leen como negativo el humor de la gente sobre su propio sufrimiento («sin café no soy persona  », ejemplo inventado), y para quien lo publica es complicidad, no queja. En un estudio real esta regla subió con claridad la precisión del negativo a cambio de muy poca cobertura.

```
QUEJA = re.compile(r"no lleg|tarde|devol|reclam|estaf|nunca|p[eé]simo|cobr", re.I)

def regla_humor(m):
    if (m["tono"] == "Negativo" and m.get("humor") and not QUEJA.search(m["limpio"])
            and not m.get("concreto")):
        m["tono"] = "Neutral"
    return m
```

El tono se mide por aspecto, no global. Una mención puede amar el café y odiar la entrega. Por eso el tablero muestra el tono dentro de cada objeto (el café, la entrega, el precio), no un solo número de sentimiento.

PASO 6

Imágenes con vocabulario cerrado

Las imágenes dicen cosas que el texto no: si el producto aparece, en qué contexto, si se ve la marca. Se leen con la misma lógica que el texto: **una lista cerrada** y salida en JSON.

```
Mira la imagen y responde SOLO con JSON.
"aparece": lista con términos de [bolsa_de_cafe, taza, cafetera, empaque_abierto,
           logo_de_la_marca, persona, texto_en_imagen]
"contexto": uno de [casa, oficina, cafeteria, exterior, no_se_sabe]
"no_mapeado": lista de cosas que ves y no están en la lista (máximo 3)
No inventes términos dentro de "aparece".
```

Lo que no encaja va a `no_mapeado` en lugar de inventar categorías. Y ahí aparece lo interesante: si «mascota» sale una y otra vez en `no_mapeado`, quizás merece ser una etiqueta en la próxima versión.

Antes de la corrida, una prueba mínima con una imagen sintética de 8×8 píxeles: confirma que el modelo responde, que la clave sirve y que el JSON llega completo, por casi nada.

```
import base64, io
from PIL import Image

buf = io.BytesIO()
Image.new("RGB", (8, 8), (120, 80, 40)).save(buf, format="PNG")
parte_imagen = {"inline_data": {"mime_type": "image/png",
                                "data": base64.b64encode(buf.getvalue()).decode()}}
cuerpo = {"contents": [{"role": "user", "parts": [parte_imagen, {"text": PROMPT_IMAGEN}]}],
          "generationConfig": {"temperature": 0,
                               "responseMimeType": "application/json",
                               "maxOutputTokens": 1024}}
```

Una línea de `visual.jsonl`:

```
{"img": "img-0091.jpg", "aparece": ["bolsa_de_cafe", "taza"],
 "contexto": "casa", "no_mapeado": ["mascota"]}
```

> **Detalle que ahorra plata:** dale margen a la respuesta (`maxOutputTokens`). Con poco presupuesto el modelo corta el JSON a la mitad y pierdes la llamada. Leer 1.000 imágenes costó cerca de US$ 0,85 en oct-2026; las 2.000 del ejemplo, unos US$ 1,70.

¿De dónde salen las imágenes? De tus propias cuentas, de las publicaciones que te autorizaron y de la prensa. Las mismas reglas de captura del paso 2 aplican aquí.

PASO 7

Medición: ¿cuánto acierta?

Este es el paso que separa un estudio serio de uno bonito, y el que casi nadie hace, con licencia o sin ella. Se compara lo que dijo el modelo con lo que leíste a mano en el set dorado.

Las tres cifras, por clase:

- **Precisión:** de lo que el modelo marcó negativo, cuánto lo era de verdad.
- **Cobertura:** de lo que de verdad era negativo, cuánto encontró el modelo.
- **F1:** el promedio que castiga cuando una de las dos se cae.

```
import csv, math
from collections import Counter

def metricas(ruta):
    filas = list(csv.DictReader(open(ruta, encoding="utf-8")))
    clases = sorted({f["humano"] for f in filas} | {f["modelo"] for f in filas})
    print(f"{'clase':10} {'precisión':>9} {'cobertura':>9} {'F1':>5} {'n':>5}")
    for c in clases:
        vp = sum(f["humano"] == c and f["modelo"] == c for f in filas)
        fp = sum(f["humano"] != c and f["modelo"] == c for f in filas)
        fn = sum(f["humano"] == c and f["modelo"] != c for f in filas)
        p = vp / (vp + fp) if vp + fp else 0
        r = vp / (vp + fn) if vp + fn else 0
        f1 = 2 * p * r / (p + r) if p + r else 0
        n = vp + fn
        aviso = "  margen amplio" if n < 20 else ""
        print(f"{c:10} {p:9.2f} {r:9.2f} {f1:5.2f} {n:5}{aviso}")
    acuerdo = sum(f["humano"] == f["modelo"] for f in filas) / len(filas)
    h, m = Counter(f["humano"] for f in filas), Counter(f["modelo"] for f in filas)
    azar = sum(h[c] * m[c] for c in clases) / len(filas) ** 2
    print(f"kappa: {(acuerdo - azar) / (1 - azar):.2f}")

metricas("tono.csv")     # columnas: humano, modelo
```

Así se ve (cifras de ejemplo):

```
clase      precisión cobertura    F1     n
negativo        0.88      0.81  0.84    96
neutral         0.86      0.90  0.88   162
positivo        0.91      0.89  0.90   142
kappa: 0.79
```

Cómo leerlo:

- **La meta es 0,85** en relevancia y en las clases que sostienen un titular. Si la marca va a decir «el negativo sobre la entrega subió», el negativo tiene que pasar la vara.
- El tono se mide solo sobre las menciones que el dorado y el modelo dan por relevantes.
- Las clases con menos de 20 casos se publican igual, con su advertencia de margen amplio.
- **Si no llega, no se baja la vara:** se corrige el «NO es» de las etiquetas que se confunden y se vuelve a medir con la mitad de ajuste del dorado.

PASO 8

Informe que se verifica solo

La regla que más me ha protegido: **las cifras las calcula el código y la IA redacta sobre ellas**. Nunca al revés. Si le pides a un modelo que cuente, a veces cuenta mal con mucha seguridad.

```
import pandas as pd

et = pd.read_json("etiquetas.jsonl", lines=True)
et = et[et["rel"]]
et["semana"] = pd.to_datetime(et["fecha"]).dt.isocalendar().week
tabla = (et.groupby(["objeto", "tono"]).size().unstack(fill_value=0))
tabla["% negativo"] = (100 * tabla["Negativo"] / tabla.sum(axis=1)).round(0)
tabla.to_csv("cifras.csv")
```

Después, la IA recibe `cifras.csv` y una instrucción: «redacta tres hallazgos usando solo estas cifras, en lenguaje humano». Y un verificador revisa que **cada número del texto exista en las cifras**. Si una frase dice algo que los datos no sostienen, el informe no sale.

```
import re

def verificar(informe, cifras_validas):
    malas = []
    for numero in re.findall(r"\d+(?:[.,]\d+)?", informe):
        if numero.replace(".", "").replace(",", ".") not in cifras_validas:
            malas.append(numero)
    return malas      # si la lista no está vacía, el informe no sale
```

`cifras_validas` es el conjunto de números de `cifras.csv` escritos como texto (y los porcentajes ya redondeados). Es un verificador simple: se le escapan frases como «la mayoría», que se revisan leyendo. Pero atrapa lo más grave: un número que nadie puede reconstruir.

Tres reglas de forma:

- **En lenguaje humano:** «el negativo sobre la entrega sube 60 %», nunca «índice 160».
- **Cada hallazgo va al área que puede actuar**, con un plazo: la entrega a logística, el precio a comercial, el sabor a producto. La escucha termina cuando la mención llega a quien puede hacer algo con ella.
- **La hoja de vida del estudio** va con cada entrega. Es lo primero que pide quien ve el tablero.

PLANTILLA · HOJA-DE-VIDA.MD

pregunta: ¿De qué hablan quienes compran café en línea en Bogotá? ventana: 1-jul al 30-sep-2026 fuentes: YouTube (API, 46 videos), X (API, búsqueda «café en línea» y variantes), reseñas propias embudo: 31.200 capturadas, 20.150 en el análisis (ver embudo.json) taxonomía: versión 1.0 · 4 pilares · 14 etiquetas modelo: Gemini Flash, temperatura 0, sin razonamiento, lotes de 25 dorado: 400 menciones, 2 lectores, kappa 0,81 entre lectores métricas: relevancia F1 0,89 · negativo F1 0,84 (ver metricas.json) costo: US$ 6,10 texto · US$ 1,70 imágenes · US$ 0,60 sondas límites: no mide recordación ni ventas; no es una muestra de la población datos: autores anonimizados; sin capturas de redes que no lo permiten

(Cifras de ejemplo, de la tienda ficticia.)

Cuánto cuesta el ejemplo completo

Parte	Volumen	Costo
Captura de YouTube	46 videos, unos 25.000 comentarios	US$ 0 (dentro de la cuota)
Captura de X	5.000 posts	US$ 25
Clasificar texto	20.000 menciones	~US$ 6
Segunda pasada (lo que el filtro descartó)	~3.000 menciones	~US$ 1
Leer imágenes	2.000	~US$ 1,70
Sondas y validaciones	3 rondas	< US$ 1 por ronda
Lectores de tono abiertos	todo	US$ 0 (en tu computador)
Total		**~US$ 35**

El grueso es X. Sin X, el estudio completo cuesta menos de US$ 10. Costos observados en oct-2026; los precios cambian, así que haz la cuenta con los del día.

Errores que ya vi (para que no los repitas)

Error	Cómo se ve	Qué hacer
Preguntar «¿hablan mal de nosotros?»	Un estudio que solo cuenta quejas	Encuadre con fase y supuestos (paso 1)
Mezclar comercio electrónico con conversación	La marca «más querida» es la que más vende en línea	Niveles de uso en el embudo (paso 3)
Tirar la capa que no entró al corte	Se pierde el hallazgo emocional	Guardarla con nombre (paso 3)
Lista abierta de temas	«Envío», «entrega», «domicilio»: conteos que no suman	Taxonomía cerrada con «NO es» (paso 4)
Creerle al sentimiento de la plataforma	Casi todo neutral	Varios lectores y regla del humor (paso 5)
Ajustar y reportar con el mismo dorado	Métricas infladas	Dorado en dos mitades (paso 4)
Correr todo sin sonda	La factura llega después	Sonda y tope aprobados antes (paso 5)
Volver a pagar lo ya etiquetado	Cada caída cuesta la corrida entera	Escritura inmediata y reanudable (paso 5)
Que la IA cuente	Cifras seguras y falsas	El código cuenta, la IA redacta, el verificador revisa (paso 8)
Extraer redes que no lo permiten	Riesgo legal y de reputación	APIs oficiales o autorización escrita (paso 2)

Lista de verificación

- ☐ Escribí el encuadre: pregunta, fase, supuestos, ventana y lo que la escucha no puede decir.
- ☐ Cada fuente tiene un trabajo claro y una forma legal de captura.
- ☐ Ningún autor viaja en claro y no hay datos personales en el entregable.
- ☐ El embudo tiene criterio, código y conteo en cada paso.
- ☐ El comercio electrónico y los medios van aparte de la conversación.
- ☐ La capa que sale del corte temático quedó guardada con nombre.

- ☐ La taxonomía es cerrada, tiene versión y cada etiqueta dice lo que NO es.
- ☐ El set dorado tiene entre 300 y 500 menciones, estratificadas y barajadas.
- ☐ Medí el acuerdo entre lectores y partí el dorado en dos mitades.
- ☐ El prompt se arma desde el JSON de la taxonomía.
- ☐ Corrí la sonda y el costo proyectado y el tope están aprobados.
- ☐ La corrida es reanudable y el gasto se calcula con los tokens facturados.

- ☐ Hice la segunda pasada sobre lo que descartó el filtro.
- ☐ El tono lo deciden varios lectores y apliqué la regla del humor.
- ☐ Las imágenes se leen con lista cerrada y `no_mapeado`.
- ☐ Precisión, cobertura y F1 llegan a 0,85 en lo que sostiene un titular.
- ☐ Las cifras las calcula el código y el verificador no encontró números sin respaldo.
- ☐ Cada hallazgo tiene un área responsable y un plazo.

- ☐ La hoja de vida del estudio va con la entrega.

De dónde sale esto

Todo lo de esta guía lo he aplicado en estudios reales de escucha social. El ejemplo de la tienda de café es **ficticio**, y las cifras de salida (embudo, métricas, hoja de vida) son de ejemplo. Los costos por unidad son los que he medido en octubre de 2026; cambian, así que haz tu cuenta con los precios del día. Los nombres de los modelos abiertos y de las APIs son los vigentes a esa fecha.

No soy programador; vengo de la estrategia. Esto lo he aprendido construyendo, así que habrá cosas que alguien de ingeniería haría distinto, y me sirve que me lo digan.

Si esta guía te sirvió, guárdala y pásasela a quien le reporta el sentimiento a tu marca.